

Desenvolvimento de Agentes com LLM · 40 horas · MBA PCDF - IA Generativa

- Disciplina/Tema : Anatomia do Agente Conversacion
- Professor(a): Yuri Lopes Queiroz



E1: Anatomia do Agente Conversacional

Desenvolvimento de Agentes com LLM · 40 horas · MBA PCDF - IA Generativa

SINARM — SEGURANÇA PÚBLICA INTELIGENTE

14/07 E 16/07/2026



 **ibmec**

O que Você Aprenderá em E1

01

Arquitetura do Agente

Entender os componentes de um agente conversacional moderno

02

Tools Profissionais

Implementar ferramentas que a LLM pode invocar autonomamente

03

Ciclo ReAct

Compreender o padrão Reasoning + Acting em agentes

04

Debug em Produção

Diagnosticar e corrigir falhas em agentes reais

05

Dados SINARM

Usar queries estruturadas em bases de segurança pública

06

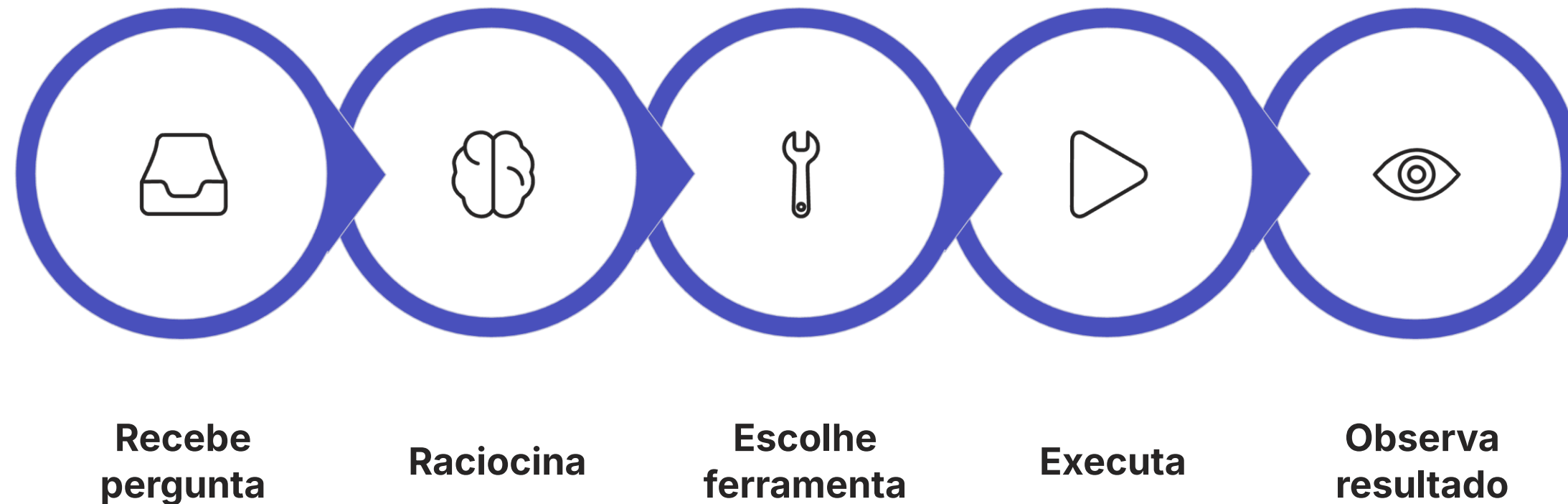
Diagnóstico de Erros

Identificar quando e por que o agente falha



Definição: Agente de IA

Um agente é um programa que **percebe, raciocina, age e itera** até resolver uma tarefa.



O ciclo se repete até o agente chegar a uma resposta confiável ou declarar falha.

Exemplo Prático

Pergunta: "Quantos roubos de Taurus houve em DF?"

Processo: Pensa → Escolhe → Executa → Observa → Responde

Resultado Verificado

"Encontrei **580 roubos** de Taurus em DF" — Fonte: SINARM, 11/07/2026

Diferença Crítica: LLM Simples vs Agente

Aspecto	LLM Simples	Agente
Entrada	Apenas texto	Texto + Ferramentas
Processamento	Uma única passada	Múltiplas iterações
Dados	Treino estático	Tempo real dinâmico
Precisão	Pode alucinar	Dados verificados
Exemplo	ChatGPT padrão	ChatGPT + Pesquisa

 LLM Simples

"Acho que havia 580..." — pode estar errado

 Agente

"Consultei o SINARM e encontrei 580..." — verificado!

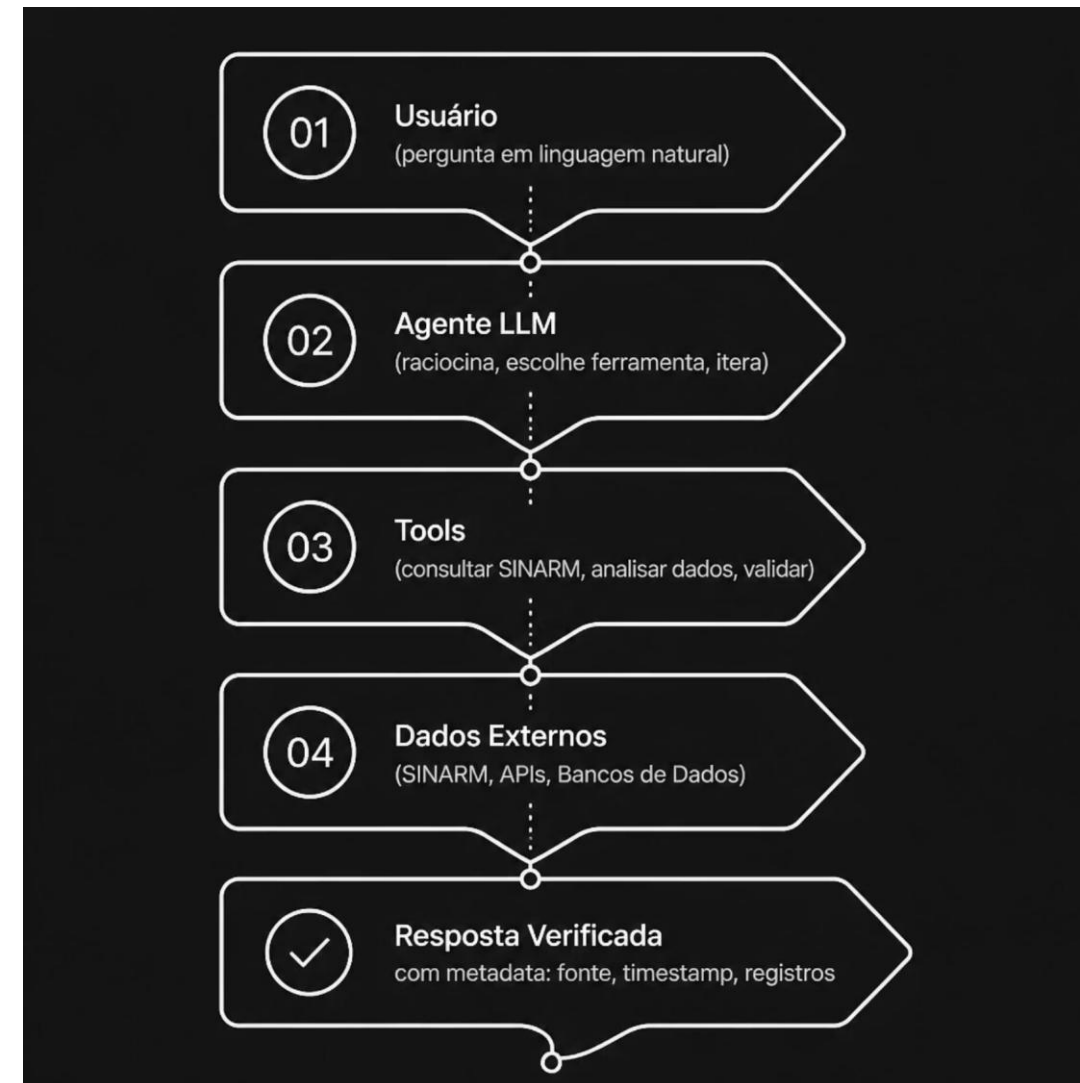


Componentes Principais da Arquitetura

O agente orquestra quatro camadas para entregar respostas verificadas e rastreáveis:

- **Usuário** — faz a pergunta em linguagem natural
- **Agente (LLM)** — raciocina, escolhe e itera
- **Tools** — executam ações reais (busca, análise, validação)
- **Dados Externos** — SINARM, APIs e bancos de dados

O resultado inclui **fonte, timestamp e metadados** para auditoria completa.



Tool vs Função Comum

Uma tool profissional é projetada para ser **lida e usada por uma LLM** — não apenas por humanos.

- 1 Assinatura Clara**
Nome descritivo + tipos bem definidos: `consultar_ocorrencias_sinarm(query: str) -> str`
- 2 Docstring Descritivo**
A LLM lê para decidir quando usar — explica O QUE faz e dá exemplos
- 3 Validação de Input**
Rejeita entradas inválidas: verifica tipos, formatos e conteúdo
- 4 Tratamento de Erro**
Try/except completo — nunca deixa o agente quebrar; mensagens claras
- 5 Output Estruturado**
Sempre JSON com estrutura consistente e metadados: fonte e timestamp



Implementação Prática — Tool SINARM

```
from langchain.tools import tool
import json
from datetime import datetime

@tool
def consultar_ocorrencias_sinarm(query: str) -> str:
    """Busca ocorrências no banco SINARM.

    Args:
        query: Formato 'campo:valor'
        Exemplos: 'marca:Taurus',
                  'tipo:Furto', 'uf:DF'

    Returns:
        JSON com registros encontrados
    """
    # VALIDAÇÃO: verificar tipo
    if not isinstance(query, str):
        return json.dumps(
            {"erro": "Query deve ser string"}
        )
```

Destaques do Código

@tool decorator

Registra a função como tool utilizável pela LLM

Docstring completa

Instrui a LLM sobre uso e exemplos válidos

Validação de tipo

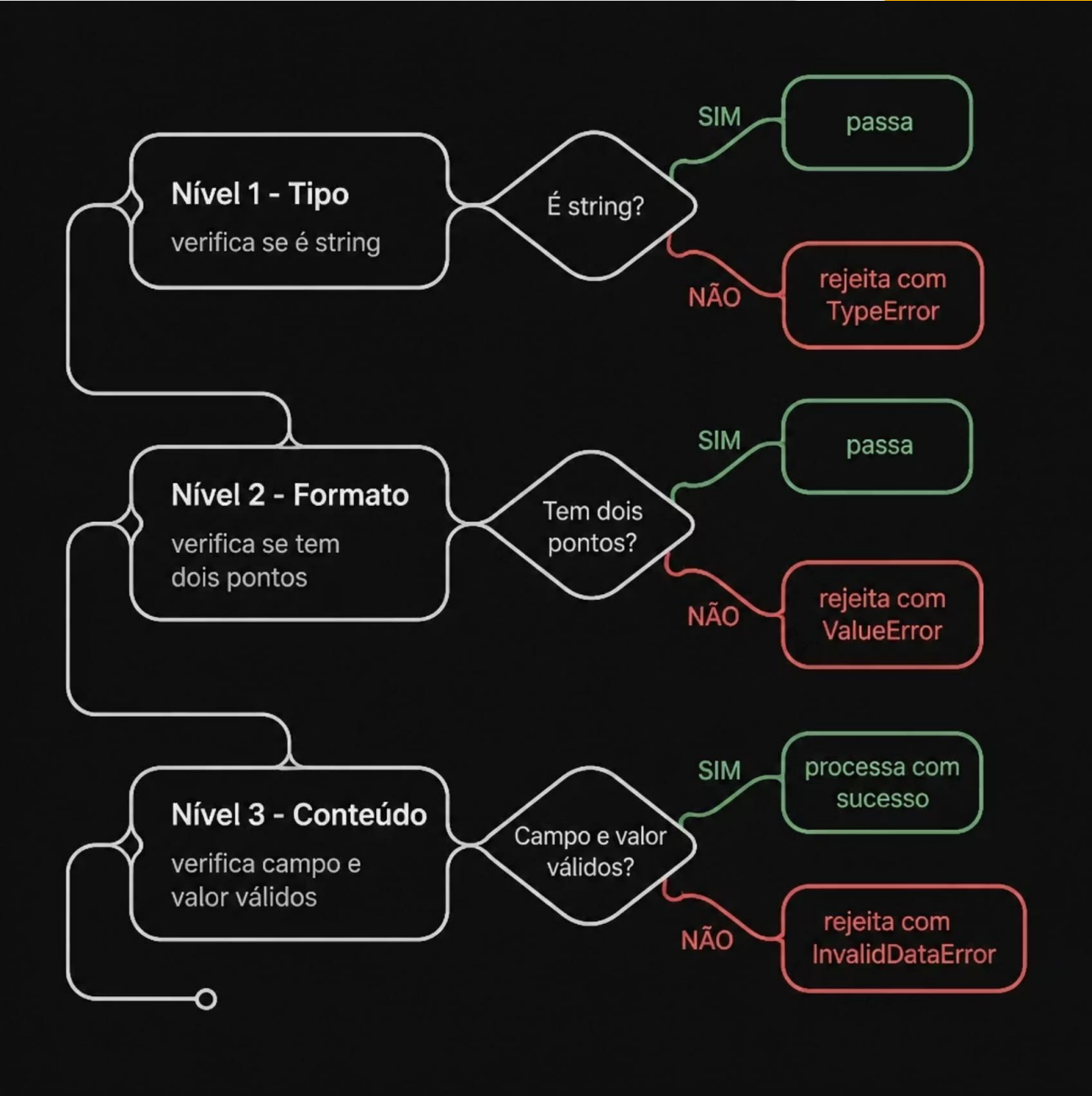
Primeira barreira contra inputs malformados



Validação em Cascata

Cada nível filtra inputs inválidos antes de alcançar a lógica principal. Se qualquer nível falhar, o agente recebe um erro descritivo — nunca uma exceção não tratada.

Nível	Valida	Por quê	Exemplo
1	Tipo de dado	Evita TypeError	<code>isinstance(x, str)</code>
2	Formato	Garante estrutura	<code>":" in query</code>
3	Conteúdo	Campo e valor válidos	<code>campo in válidos</code>



6 Partes Essenciais de uma Tool

Assinatura

Nome + tipos dos parâmetros

Output

JSON estruturado

Lógica

Processamento da ação



Docstring

Instruções para a LLM

Validação

Input verification

Try/Except

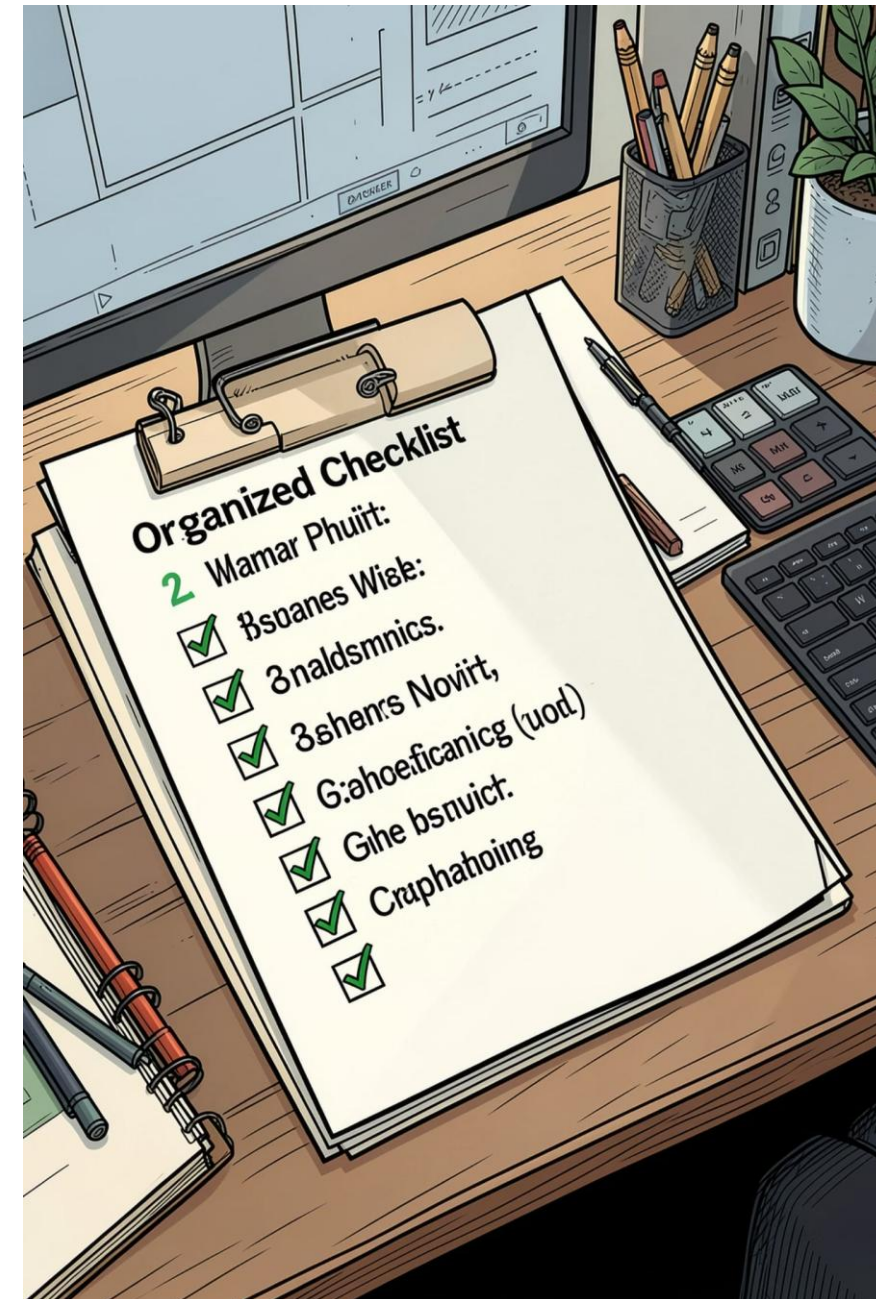
Erro handling robusto

Cada componente protege o próximo — da assinatura ao output, a tool opera de forma segura, previsível e auditável em produção.

E1 — ANATOMIA DO AGENTE CONVERSACIONAL

Recap: 6 Componentes de uma Tool

Sem tools bem construídas, o agente não funciona. Cada componente tem papel crítico no ciclo de execução.



The logo for ibmec, consisting of a small yellow circle above the word "ibmec" in a bold, blue, sans-serif font.

COMPONENTES DA TOOL

Checklist: Os 6 Componentes Essenciais

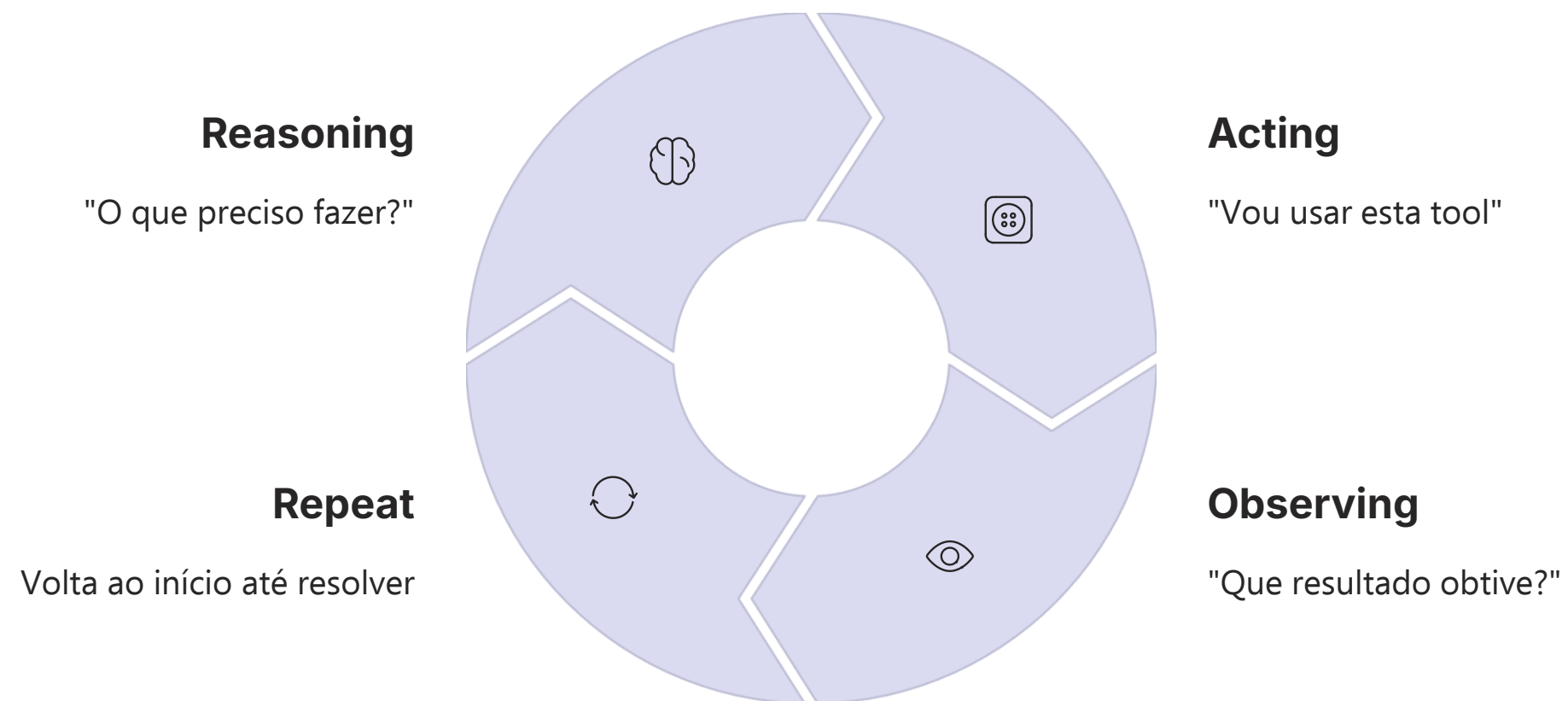
- 1 **Assinatura**
Nome claro + tipos bem-definidos
- 2 **Docstring**
Explicar O QUE e COMO usar
- 3 **Validação**
Verificar tipo, formato e conteúdo
- 4 **Try/Except**
Tratar erros graciosamente
- 5 **Lógica**
Executar o trabalho real
- 6 **Output**
Retornar JSON estruturado

❏ **Sem tools boas, o agente não funciona.** Cada componente protege o ciclo ReAct.

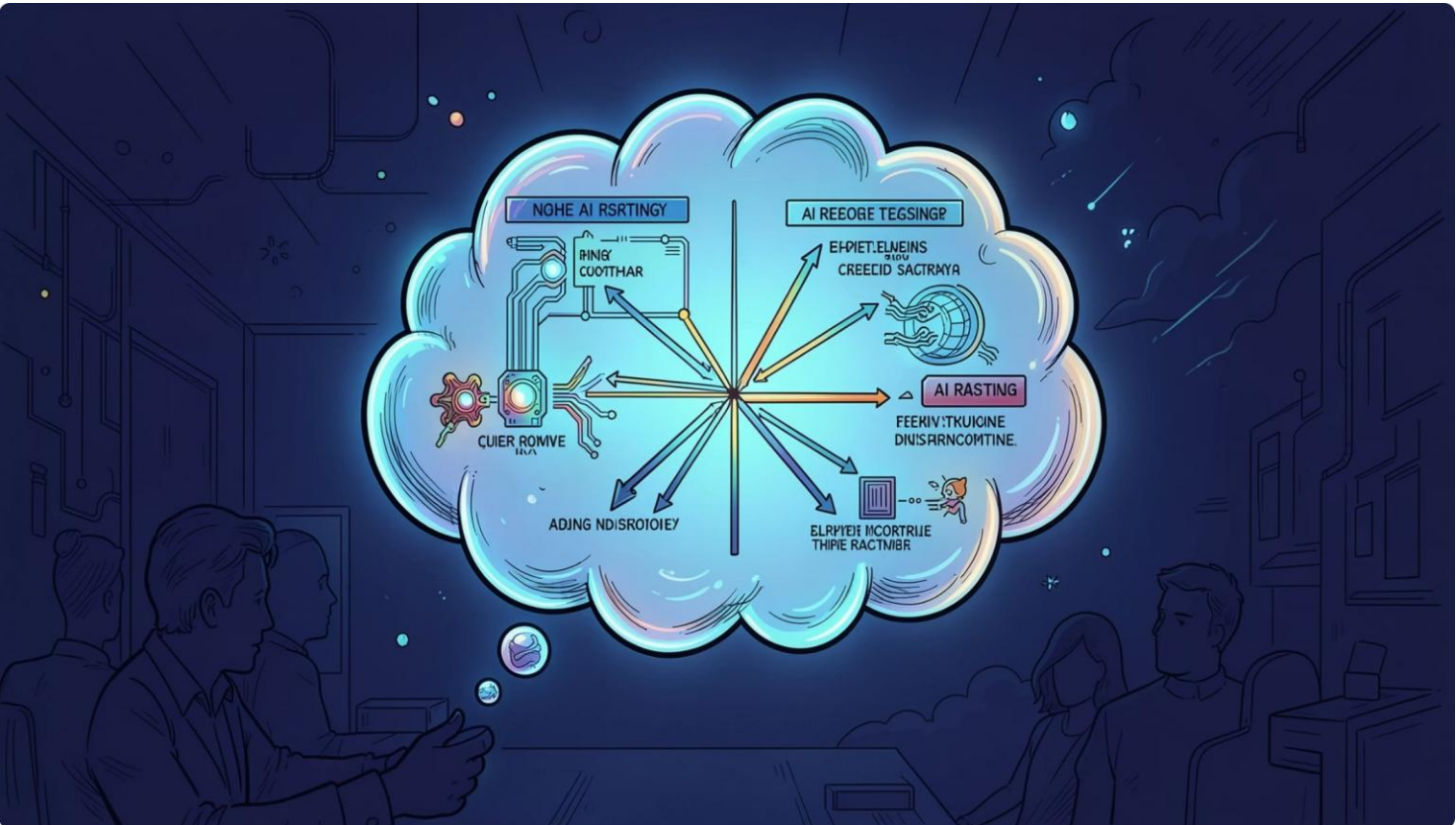


ReAct: Reasoning + Acting

O agente **alterna entre pensar e agir** até resolver a tarefa. Um ciclo iterativo com quatro etapas.



THOUGHT: Como o Agente Pensa



O agente **planeja antes de agir**: avalia a pergunta, seleciona a tool e define parâmetros.

Entendi a Pergunta

"Quantos roubos de Taurus em DF?"

Recursos Disponíveis

Tool: `consultar_ocorrencias_sinarm(query)`

Estratégia

Query: `marca:Taurus AND tipo:Roubo AND uf:DF`

Validação

Resultado deve ser ≤ 17.760 (total Taurus)



ETAPA 2 — ACTION

ACTION: Agente Executa a Tool

Formato padrão: ACTION: [TOOL_NAME] INPUT: {parametros}

✓ Simples OK

INPUT: `marca:Taurus`
→ 1 query, resultado direto

⚠ Com Erro de Formato

INPUT: `Taurus` (incompleto)
→ Agente adapta a query

✗ Input Inválido

INPUT: `None`
→ Tool rejeita, agente reconsidera

🎯 Composta Complexa

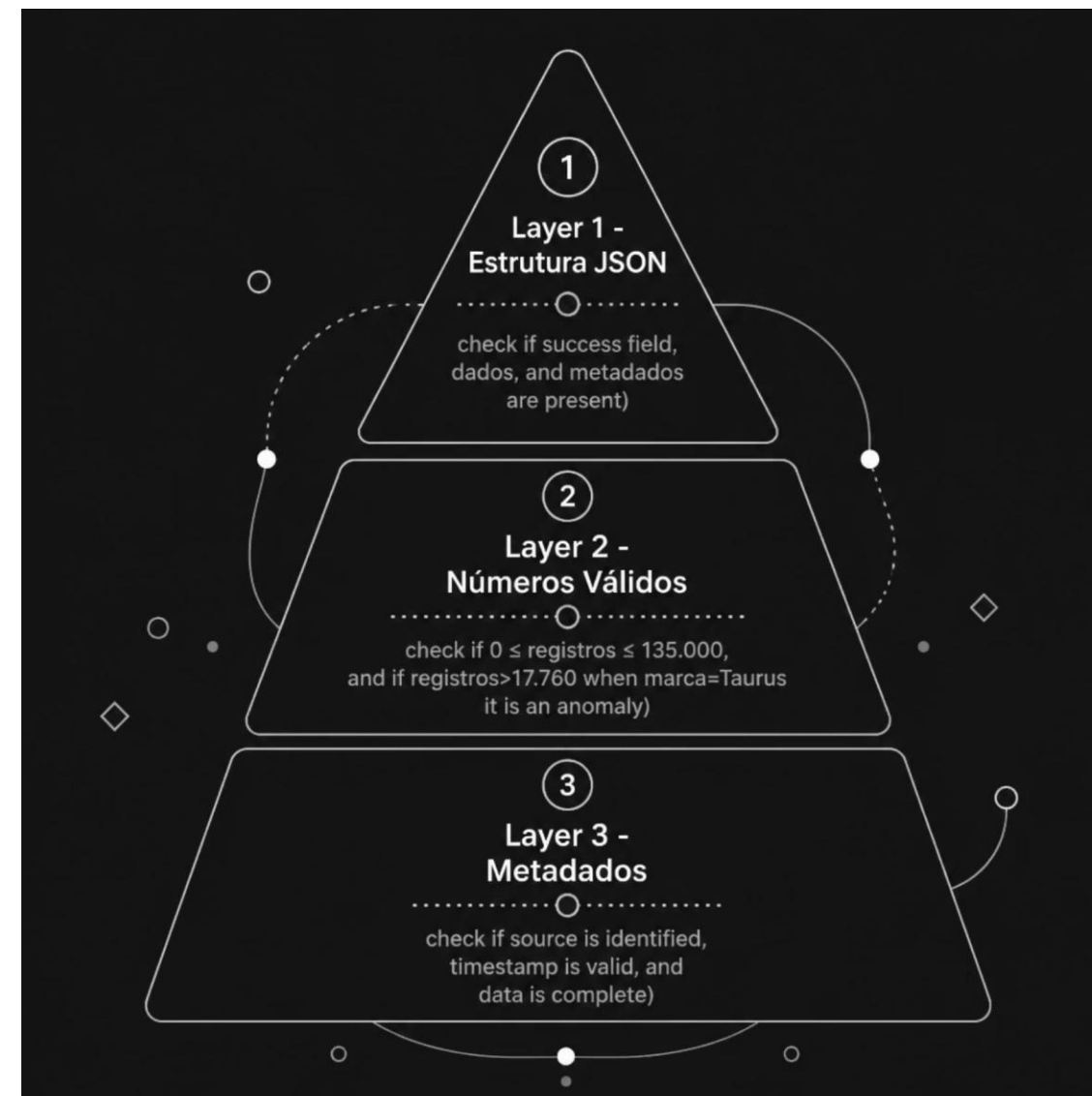
INPUT: `marca:Taurus AND tipo:Roubo AND uf:DF`
→ 1 query precisa



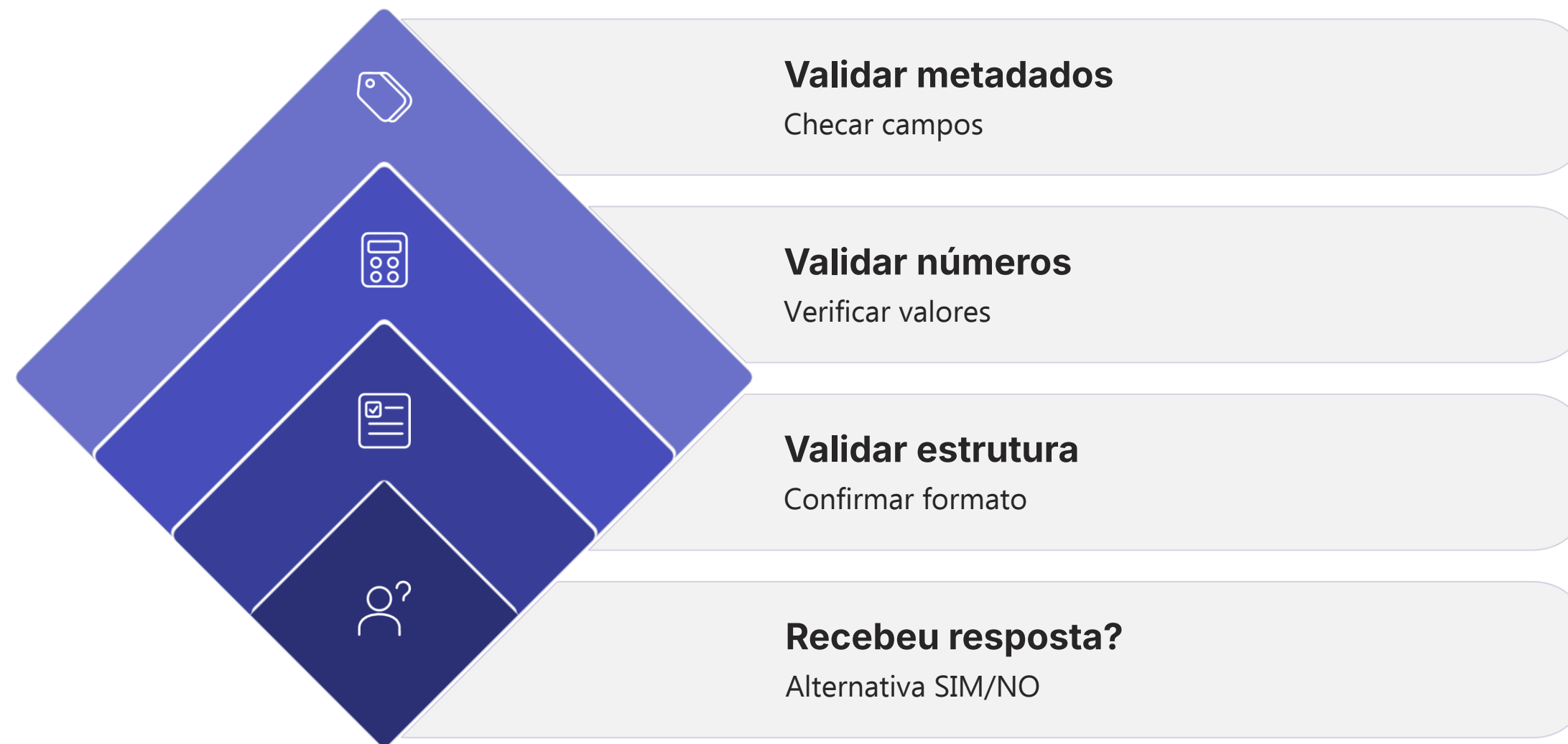
OBSERVATION: Agente Valida o Resultado

Antes de confiar nos dados, o agente valida em **três níveis**.
Se qualquer nível falhar, o resultado é questionado.

⚠ Se falha em validação, o agente **NÃO** deve confiar nos dados.



ETAPA 3 — DECISÃO

OBSERVATION: Árvore de Decisão

O agente segue uma árvore de decisão estruturada: valida estrutura → números → metadados. Qualquer falha aciona um caminho alternativo — aviso, revisão ou retry.

ibmec

CICLO COMPLETO

Ciclo ReAct: Exemplo Simples vs. Complexo

✓ Cenário Simples

Pergunta: "Quantas armas Taurus?"

1. THOUGHT: query simples
2. ACTION: `marca:Taurus`
3. TOOL: `{"registros": 17760}`
4. OBSERVATION: validado ✓
5. FINAL ANSWER: 17.760 armas

🕒 1 iteração · ~200ms

⚙️ Cenário Complexo

Pergunta: "Roubos de Taurus em DF?"

1. `marca:Taurus` → 17.760 (parcial)
2. `tipo:Roubo` → 12.000 (parcial)
3. `uf:DF` → 45.000 (parcial)
4. Combinar e calcular intersecção...

🕒 3-4 iterações · 1-2 segundos

CONTROLE DO LOOP

Quando o Loop ReAct Para?

Loop CONTINUA se...

- Pergunta **NÃO** foi respondida
- Há ferramentas disponíveis para tentar
- Não atingiu o limite (máx. 10 iterações)

Loop PARA se...

- Pergunta **FOI** respondida ✓
- Sem mais ferramentas ✕
- Timeout (10+ iterações) ⌚
- Erro crítico ✖

Proteções do Sistema

Max 10 iterações

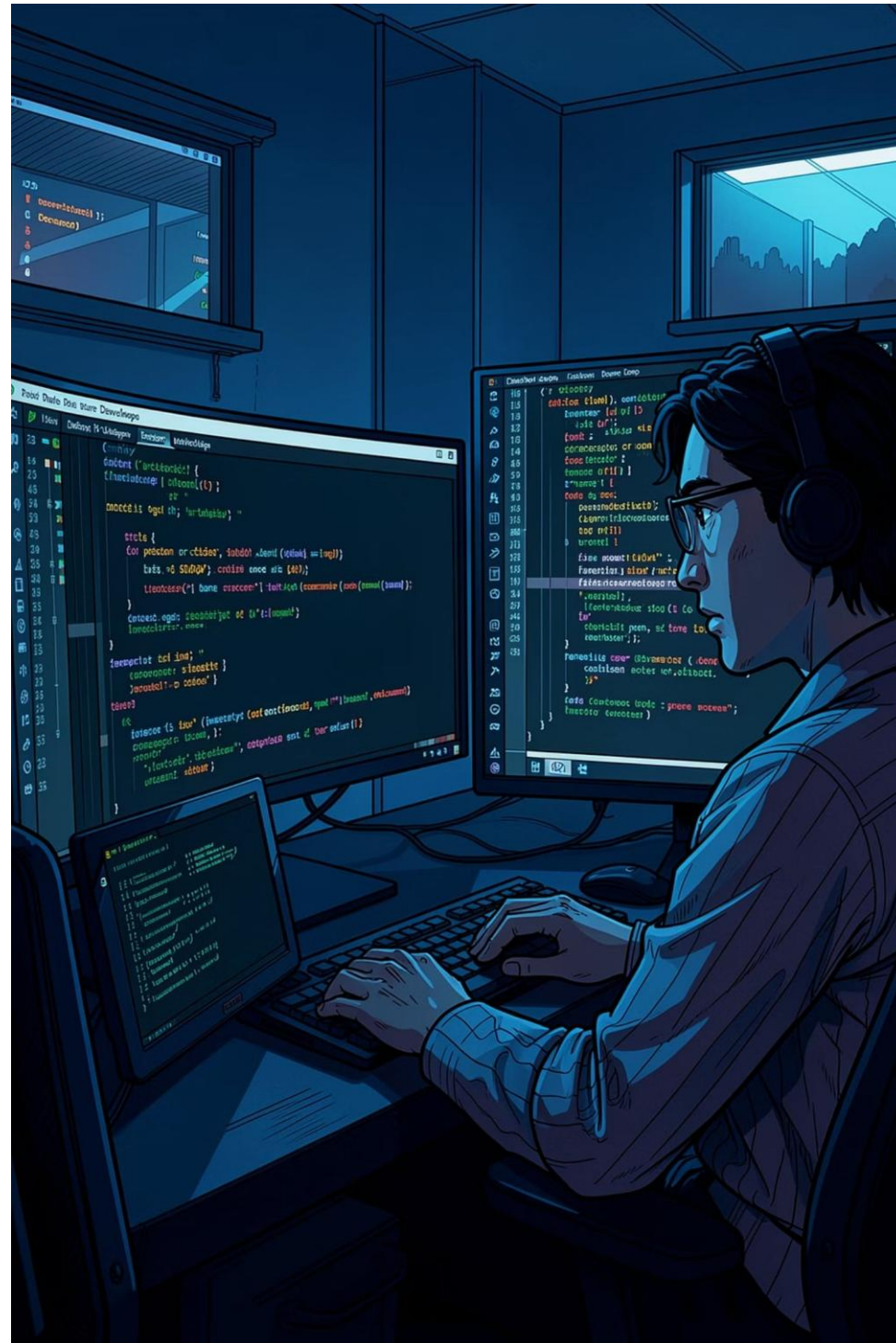
Evita loop infinito

Timeout por query

Evita travamento

Rastreamento de repetição

Detecta loops



BLOCO 3 · DEBUGGING — PARTE 1

Debugging: Encontrar e Corrigir Problemas

Em sistemas de IA Generativa com dados de segurança pública, identificar e corrigir falhas não é opcional — é obrigatório. Conheça os 5 tipos de problemas mais comuns em agentes conversacionais.



Os 5 Tipos de Problemas em Agentes

Erros (Crashes)

Tool quebra, agente trava. Causa: bug no código.

Alucinações

Agente "inventa" dados. Causa: LLM sem validação.

Loops Infinitos

Agente itera sem parar. Causa: tool design ruim.

Decisões Erradas

Agente escolhe tool incorreta. Causa: docstring vaga.

Dados Inválidos

Agente retorna dados errados. Causa: validação insuficiente.

⚠ Em SINARM (dados de segurança pública), debugging é OBRIGATÓRIO.

Por Que Debugging Importa?

O impacto de um bug varia drasticamente entre desenvolvimento e produção — especialmente em sistemas de segurança pública.

Desenvolvimento

- Bugs afetam: ~5 pessoas
- Tempo de debug: ~1 dia
- Impacto: baixo
- Risco: mínimo

Produção

- Bugs afetam: milhares de pessoas
- Tempo de debug: crítico
- Impacto: alto — segurança pública
- Risco: máximo

Em SINARM, debugging em produção é essencial — não é uma etapa opcional do ciclo de vida.



Verbose Mode: Visibilidade Total

O **Verbose Mode** transforma o agente de "caixa preta" em "caixa transparente", revelando cada etapa do raciocínio.

✗ Dark Box (Sem Verbose)

Input: "Quantos roubos de Taurus em DF?"

Output: "580 roubos encontrados"

✗ Não sabemos como chegou nesse número!

✓ Glass Box (Com Verbose)

```
[AGENTE] THOUGHT: Preciso de 3 critérios
[AGENTE] ACTION:
consultar_ocorrencias_sinarm(...)
[TOOL] Validação: OK ✓
[TOOL] Busca em SINARM...
[TOOL] Encontrado: 580 registros
[AGENTE] OBSERVATION: Validando...
[AGENTE] Números fazem sentido? ✓
[AGENTE] FINAL ANSWER: 580 roubos
```



Como Implementar Verbose Mode

```
import logging

# Configurar logging
logging.basicConfig(
    level=logging.DEBUG,
    format='%(name)s] %(levelname)s: %(message)s'
)
logger = logging.getLogger('agente_sinarm')

class Agente:
    def processar(self, pergunta, verbose=False):
        if verbose:
            logger.info(f"Iniciando: {pergunta}")

        # THOUGHT
        thought = self._think(pergunta)
        if verbose:
            logger.debug(f"THOUGHT: {thought}")

        # ACTION
        action = self._act(thought)
        if verbose:
            logger.debug(f"ACTION: {action}")

        # OBSERVATION
        observation = self._observe(action)
        if verbose:
            logger.debug(f"OBSERVATION: {observation}")

        # FINAL ANSWER
        answer = self._answer(observation)
        if verbose:
            logger.info(f"FINAL ANSWER: {answer}")
        return answer
```

3 Passos Essenciais

1. Importar o módulo `logging`
2. Configurar `basicConfig` com nível e formato
3. Adicionar `logger.debug / info / warning` em cada etapa do agente

O parâmetro `verbose=False` permite ativar ou desativar os logs sem alterar a lógica principal do agente.



Formato Padrão de Logs

Estrutura do Log

```
[TIMESTAMP] [COMPONENTE] [NÍVEL] Mensagem

2026-07-11T14:30:15.234Z [AGENTE] INFO Iniciando
2026-07-11T14:30:15.450Z [TOOL] DEBUG Validação
OK
2026-07-11T14:30:15.600Z [TOOL] ERROR Campo
inválido
```

5 Níveis de Log

Nível	Quando	Exemplo
DEBUG	Detalhes técnicos	Campo validado ✓
INFO	Importante	Consulta bem-sucedida
WARNING	Suspeito	0 registros encontrados
ERROR	Recuperável	Campo inválido
CRITICAL	Grave	SINARM offline!



Onde Guardar os Logs?

A escolha do armazenamento impacta auditabilidade, desempenho e conformidade com a LGPD.

Arquivo ✓ Recomendado

`/logs/sinarm_agente_2026_07_11.log`

- Persistente e auditável
- Cresce com o tempo
- Guardar mínimo 6 meses (LGPD)

Console

Stdout durante testes

- Visível em tempo real
- Se o app cai, perde os logs

Banco de Dados ✓ Recomendado

Tabela `logs` (id, timestamp, componente, mensagem)

- Queryável e pesquisável
- Overhead adicional

 **Recomendação:** Use Arquivo + Banco de Dados para redundância e conformidade com LGPD.



Detectando Loops Infinitos

Problema Real

```
Iteração 1: ACTION "tipo:Ilegal" → 0 registros  
Iteração 2: ACTION "tipo:Ilegal" → 0 registros (REPETIU!)  
Iteração 3: ACTION "tipo:Ilegal" → 0 registros (REPETIU!)  
...  
Iteração 100: TIMEOUT – travou!
```

Solução: Rastreamento

```
iteracoes = []  
max_iteracoes = 10  
  
for iter in range(max_iteracoes):  
    action = agente.act()  
    if action in iteracoes:  
        logger.error("LOOP INFINITO DETECTADO!")  
        break  
    iteracoes.append(action)
```

- Máximo de 10 iterações
- Timeout por query
- Rastreamento de repetição



Erros Comuns em Python

TypeError

Tipo errado. Ex: `int.split()`. Fix: validar tipo.

ValueError

Valor inválido. Ex: `int("abc")`. Fix: validar antes de converter.

MemoryError

RAM esgotada. Ex: `df.copy()` em 30GB. Fix: processar em chunks.

KeyError

Chave inexistente. Ex: `df['xyz']`. Fix: listar chaves válidas.

IndexError

Índice fora do range. Ex: `lista[10]` em lista de 5. Fix: verificar comprimento.

TimeoutError

Operação lenta. Ex: query SQL grande. Fix: adicionar índices.

Erro	Tipo	Ação
ValueError / KeyError	Esperado	Erro amigável / listar válidos
MemoryError / TimeoutError	Crítico	Log + admin / Retry com timeout
Exception	Inesperado	Log + stack trace



E1 · BLOCO 4 · INTEGRAÇÃO FINAL

Integrando Tudo: Um Caso Real com SINARM

Neste bloco final, integramos todos os componentes do agente conversacional em um caso real: **142 roubos de armas em DF nos últimos 30 dias**. O fluxo ReAct completo — do THOUGHT inicial à resposta final — é executado com validação em 3 camadas, tratamento de erros com fallback e monitoramento em tempo real. Ao final, um quiz valida seu aprendizado.

**ibmec**

CASO DE USO COMPLETO

Fluxo ReAct: Da Pergunta à Resposta

Cenário: "Qual foi o padrão de roubos de armas em DF nos últimos 30 dias?" O agente executa o ciclo ReAct completo com duas chamadas de tool para construir uma resposta fundamentada em dados.

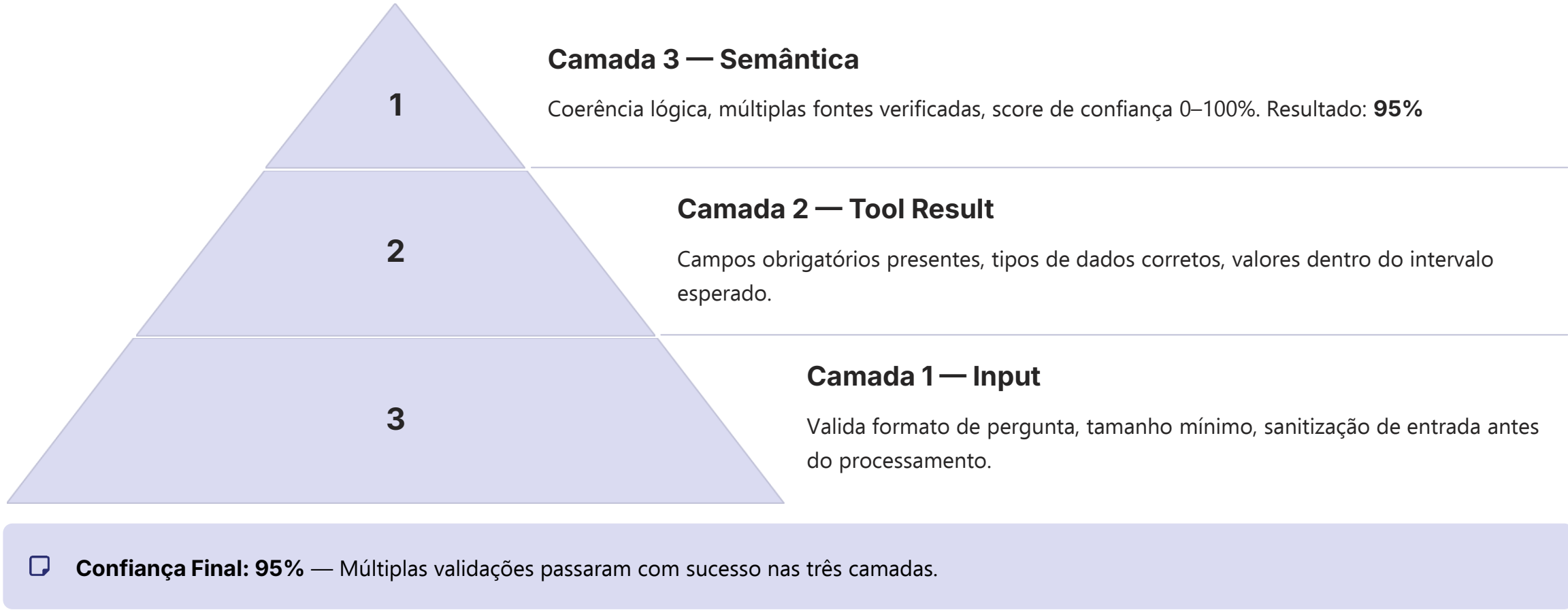
01	02
THOUGHT	ACTION 1
Identifica parâmetros: localidade (DF), tipo (roubo), item (arma), período (30 dias).	Tool: consultar_ocorrencias_sinarm com params filtrados.
03	04
OBSERVATION 1	REASONING
Retorna: total=142, bairro_top="Águas Claras", tendencia="estavel"	Dados válidos ✓, mas precisa de distribuição semanal para completar a análise.
05	06
ACTION 2	FINAL ANSWER
Tool: consultar_tendencias_sinarm — semana 1: 40, semana 2: 35, semana 3: 38, semana 4: 29.	"Em DF, 142 roubos de armas em 30 dias. Tendência ESTÁVEL. Pico: Águas Claras (18 casos). Padrão: ~35/semana, reduzindo na semana 4."



QUALIDADE E CONFIABILIDADE

Validando Respostas: 3 Camadas de Segurança

Cada resposta do agente passa por três camadas de validação antes de chegar ao usuário, garantindo integridade, coerência e confiança mensurável.



Quando Algo Dá Errado: Estratégias de Fallback

Em produção, tools falham. O agente deve lidar com erros de forma elegante, garantindo continuidade do serviço e transparência para o usuário.

Implementação Try/Except

```
try:
    resultado = tool_sinarm.consultar(query)
except SINARMOfflineError:
    logger.error("SINARM offline")
    resultado = cache.get_24h(query)
    resultado["tipo"] = "CACHE"
except:
    logger.critical("Erro desconhecido!")
    return "Desculpe, erro ao processar"
```

4 Cenários de Fallback

- **Dados indisponíveis** → Cache com warning de backup
- **Tool com timeout** → Retry com backoff exponencial
- **Limite de rate** → Fila de espera automática
- **Erro crítico** → Mensagem amigável ao usuário

⚠ Sempre registre o tipo de fallback no log para auditoria e análise de causa-raiz.

OBSERVABILIDADE

Dashboard de Saúde do Agente

Monitoramento contínuo das métricas operacionais garante detecção precoce de problemas e manutenção dos SLAs definidos para o agente em produção.

93.2%

Taxa de Sucesso

Objetivo: >95% ▲ Abaixo do esperado.
Investigar `tool_sinarm`.

1.8s

Tempo Médio de Resposta

Objetivo: <2s ✓ Dentro do esperado.

87.3%

Cache Hit Rate

Objetivo: >80% ✓ Excelente desempenho.

3

Erros por Hora

Objetivo: <5 ✓ Status normal.

99.8%

Disponibilidade SINARM

Email: 92% ▲ Intermitente. Investigar.



BOAS PRÁTICAS

Documentando seu Agente: Requirements

Documentação clara é essencial para manutenção, auditoria e handoff entre equipes. Um README bem estruturado cobre os 5 aspectos fundamentais do agente.

- 1 Purpose (O QUÊ)**
Responder perguntas sobre roubos e segurança usando dados SINARM em tempo real no DF.
- 2 Scope (ONDE FUNCIONA)**
Roubos, furtos, assaltos. Brasília (DF). Últimos 180 dias. Máx. 1000 registros por query.
- 3 Constraints (LIMITAÇÕES)**
SINARM offline → cache 24h. Rate: 100 queries/hora. Timeout: 5s. Memória: máx. 2GB.
- 4 Dependencies (DEPENDÊNCIAS)**
SINARM API (crítica), Redis cache (crítica), Email service (opcional).
- 5 Performance SLA**
Uptime: 99,5%. Latência P95: 2s. Taxa de sucesso: 95%+. Recovery: <5 min.

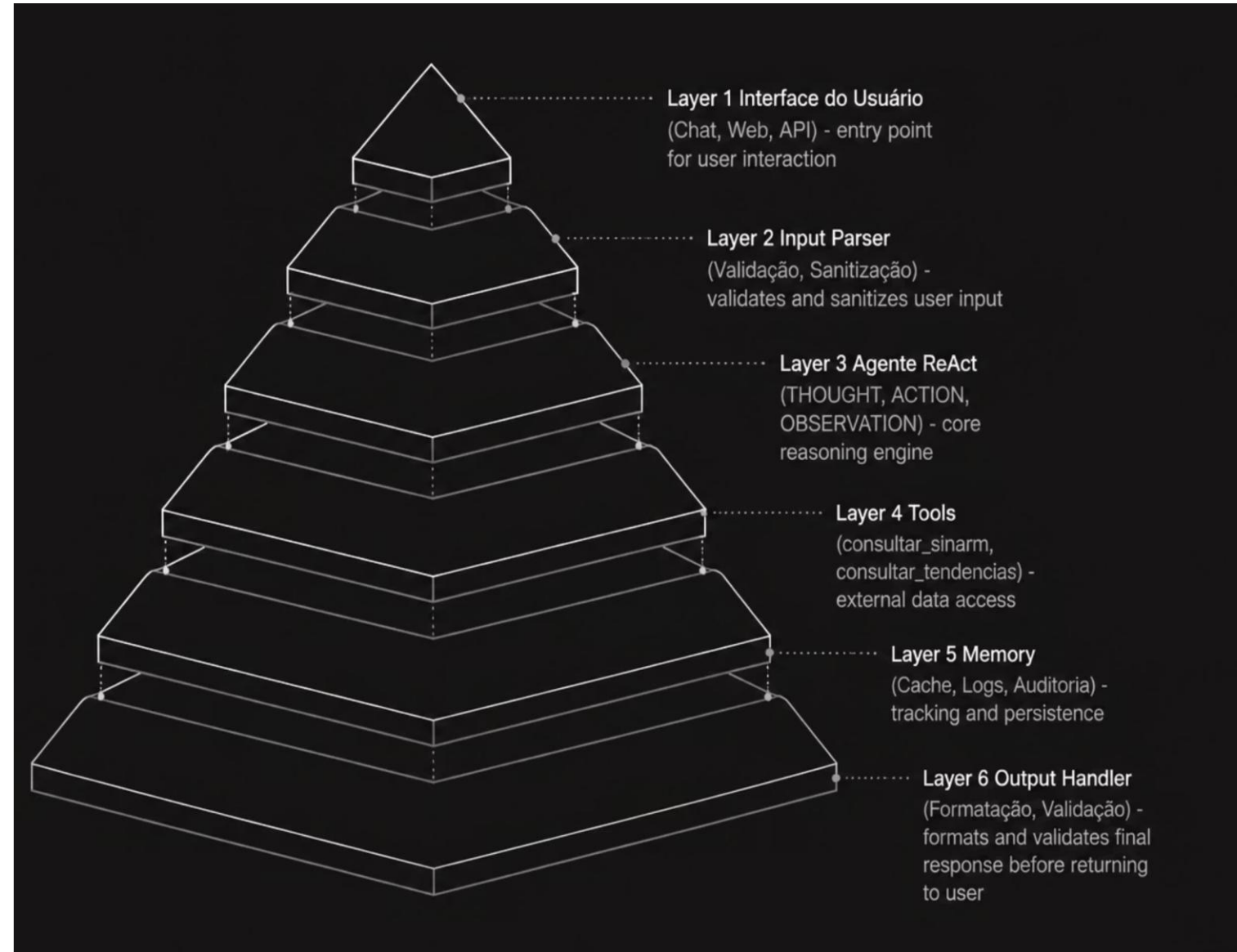


Anatomia do Agente: As 6 Camadas

O agente conversacional é composto por 6 camadas interdependentes, cada uma com responsabilidade clara. O fluxo é **bidirecional** e sempre rastreável via **Verbose Mode**.

O fluxo completo segue:

User → Interface → Parser → Agente → Tools → Memory → Output → User, com rastreamento em cada transição via logs estruturados.



AUTO-AVALIAÇÃO

Checkpoint Encontro 1: Você Entendeu?

Revise os 5 conceitos centrais do Encontro 1. Se você consegue explicar cada um e aplicá-los em código, está pronto para o E2.

1 ReAct Cycle

THOUGHT → ACTION → OBSERVATION → REASONING. Core do agente inteligente.

2 Tools

Extensões do agente que acessam dados reais. Devem ser bem documentadas, validadas e rastreadas.

3 Verbose Mode

Glass box: visibilidade total do raciocínio. Essencial para debugging e auditoria em produção.

4 Validação em 3 Camadas

Input → Tool Result → Semântica. Três barreiras de segurança antes de entregar a resposta.

5 Monitoramento

Logs estruturados, métricas em tempo real e alertas automáticos para saúde do agente.

